

Fuel Route Optimization API

Technical Design & Implementation Plan

1. Objective

Build a Django REST API that accepts a starting location and destination within the USA and returns:

1. The driving route between the two locations.
2. A map-compatible representation of the route.
3. Recommended fuel stops along the route.
4. The fuel price at each recommended station.
5. The amount of fuel purchased at each stop.
6. The total fuel cost for the journey.

The vehicle assumptions are:

- Maximum range: **500 miles**
- Fuel efficiency: **10 miles per gallon**
- Effective maximum fuel capacity: **50 gallons**
- Fuel prices come from the supplied CSV.
- Fuel-stop optimization primarily minimizes fuel expenditure.
- Fuel stations may be selected within a configurable corridor around the route.

The implementation should minimize calls to external APIs. The target is **one routing API call per request**.

2. Recommended Technology Stack

Backend

- Python
- Django **6.1.1**
- Django REST Framework
- SQLite for local development / assignment deployment
- PostgreSQL can be used if a production-oriented deployment is desired

Django 6.1.1 is currently the latest official Django release.

Geospatial processing

- Shapely
- PyProj

These will be used for local geometry calculations instead of making external API requests for every fuel station.

Routing

OpenRouteService (ORS) will be used for driving directions.

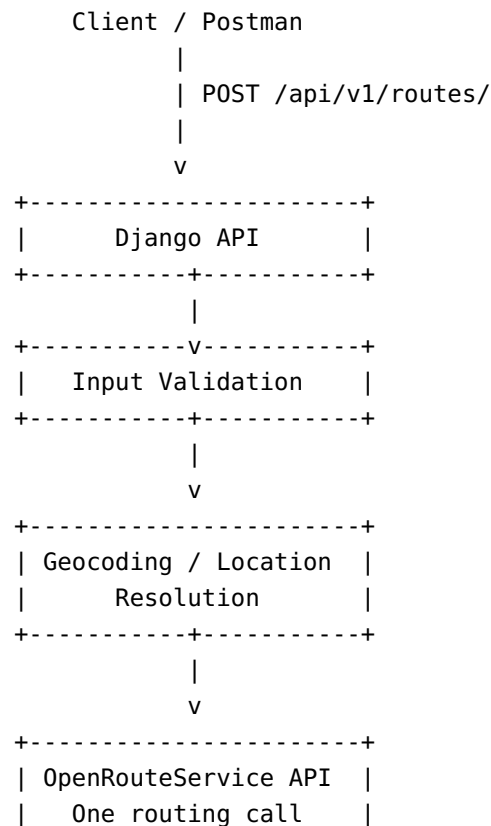
Its Directions API supports POST requests and can return GeoJSON route geometry, which is ideal for this project.

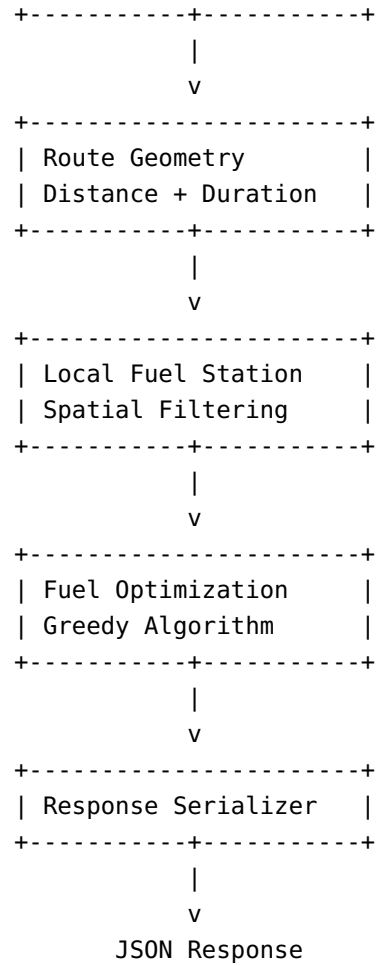
The API's documented driving-distance restriction is 6,000 km, which is sufficient for normal US trips within the scope of this assignment.

Optional

- `httpx` for external HTTP requests
- `python-dotenv` for environment variables
- `pytest` / Django test framework for testing

3. High-Level Architecture





The important architectural decision is that **the routing API is external, while fuel-station searching and optimization are local operations.**

4. Why the Fuel Dataset Should Be Imported

The supplied CSV contains fewer than 10,000 fuel stations and has these columns:

```

OPIS Truckstop ID
Truckstop Name
Address
City
State
Rack ID
Retail Price
  
```

The dataset does not contain latitude and longitude.

Therefore, the data should be processed once before the API is used.

```
fuel_prices.csv
|
v
Import / preprocessing script
|
v
Geocode station addresses
|
v
Store coordinates + price
|
v
Database
```

The API should **never geocode the fuel stations during a normal request**.

Doing so would make the API slow and would potentially require thousands of external requests.

5. Fuel Station Database Model

A Django model should be created for the supplied dataset.

```
class FuelStation(models.Model):
    opis_id = models.BigIntegerField(unique=True)

    name = models.CharField(max_length=255)

    address = models.CharField(max_length=500)
    city = models.CharField(max_length=100)
    state = models.CharField(max_length=2)

    rack_id = models.CharField(
        max_length=100,
        blank=True,
        null=True
    )

    retail_price = models.DecimalField(
        max_digits=6,
        decimal_places=3
    )
```

```
latitude = models.FloatField()
longitude = models.FloatField()

created_at = models.DateTimeField(auto_now_add=True)
```

The `OPIS Truckstop ID` should be treated as the unique station identifier.

`Retail Price` is the field used by the optimization algorithm.

`Rack ID` is retained as useful source metadata but is not required for route optimization.

6. Data Import Process

Create a Django management command:

```
python manage.py import_fuel_data fuel_prices.csv
```

The command should:

1. Read the CSV.
2. Validate required columns.
3. Normalize addresses.
4. Parse fuel prices.
5. Geocode unique addresses.
6. Store latitude and longitude.
7. Insert/update the database.

Example:

```
CSV
|
+-- Truckstop ID
+-- Name
+-- Address
+-- City
+-- State
+-- Retail Price
|
v
Normalize
|
v
```

```
Geocode unique addresses
|
v
FuelStation database
```

Important optimization

Do not geocode duplicate addresses repeatedly.

Create a normalized address key:

```
address_key = (
    f"{address}, {city}, {state}, USA"
).lower().strip()
```

Maintain a local mapping:

```
address_key -> latitude, longitude
```

This can significantly reduce the number of geocoding requests if multiple records refer to the same location.

7. Geocoding Strategy

There are two separate geocoding problems:

A. Fuel station coordinates

These should be resolved during dataset preprocessing.

B. User's start and finish locations

These need to be resolved when an API request is received.

The routing provider can potentially be used for location resolution, or a separate geocoding service can be used.

The key requirement is:

Do not geocode the fuel-station dataset on every API request.

The resulting API request should only perform location resolution for the two user-supplied locations and one route calculation.

8. API Contract

Endpoint

```
POST /api/v1/route/
```

Request

```
{
  "start": "New York, NY",
  "finish": "Chicago, IL"
}
```

Optionally, the API can later accept coordinates:

```
{
  "start": {
    "latitude": 40.7128,
    "longitude": -74.0060
  },
  "finish": {
    "latitude": 41.8781,
    "longitude": -87.6298
  }
}
```

For the assignment, accepting addresses is sufficient.

9. Request Validation

The API should validate:

- Both locations are provided.
- Both locations can be resolved.
- Both locations are within the USA.
- Start and finish are not identical.

- A route exists between them.

Example error:

```
{
  "error": {
    "code": "INVALID_LOCATION",
    "message": "Unable to resolve the supplied start location."
  }
}
```

10. Routing API

Use OpenRouteService's driving directions endpoint.

The request should contain:

```
start coordinates
finish coordinates
```

and request GeoJSON output.

The routing response provides:

- Route geometry
- Distance
- Duration

OpenRouteService's Directions API officially supports GeoJSON responses.

The desired flow is:

```
Start
|
v
OpenRouteService
|
v
Driving Route
|
+-- Distance
```

```
+-- Duration
+-- GeoJSON geometry
```

The external routing API should be called **once** for a normal request.

11. Why We Should Not Ask the Routing API About Every Fuel Station

A naïve implementation might do:

```
Get route

Station 1 -> routing API
Station 2 -> routing API
Station 3 -> routing API
...
Station 9,999 -> routing API
```

This would be slow, expensive, and would violate the assignment's requirement.

Instead, the route returned by OpenRouteService is treated as a geometric line.

Fuel stations are processed locally.

```
Route
-----

  ● Station A
    ● Station B
      ● Station C
```

The distance between the station and the route can be calculated locally.

12. Route Corridor

A fuel station does not need to be exactly on the route.

A configurable corridor should be used.

Recommended initial value:

```
ROUTE_CORRIDOR_MILES = 5
```

A station is considered a candidate if it is within approximately five miles of the route.

This provides a reasonable balance between:

- Fuel availability
- Detour distance
- Number of candidate stations

The value should be configurable rather than hardcoded throughout the application.

13. Spatial Processing

Represent the route as a Shapely `LineString`.

Conceptually:

```
route = LineString(route_coordinates)
```

Represent each fuel station as:

```
station = Point(
    station.longitude,
    station.latitude
)
```

Then calculate the station's proximity to the route.

However, latitude/longitude coordinates are angular coordinates and should not be treated as ordinary Cartesian miles.

Therefore, use an appropriate projected coordinate system through PyProj before performing distance calculations.

Conceptually:

```
WGS84 coordinates
  |
  v
Projected coordinates
  |
  v
Accurate local distance calculations
```

This is important for reliable route/station matching.

14. Calculating a Station's Position Along the Route

Knowing that a station is near the route is not enough.

We also need to know **where along the route** the station is located.

For each candidate station:

```
Route
-----
0                                     800 miles

      A           B           C
      |           |           |
    150         330         510
```

Store:

```
Station A -> 150 miles from start
Station B -> 330 miles from start
Station C -> 510 miles from start
```

This transforms the geographic problem into a one-dimensional optimization problem.

The optimizer now only needs:

```
route_position
price
```

15. Candidate Station Structure

Internally, candidates can look like:

```
{
  "station": station,
  "route_position_miles": 330.2,
  "distance_from_route_miles": 1.4,
  "price_per_gallon": 3.19
}
```

After processing all stations:

Station A	position=150	price=3.50
Station B	position=330	price=3.19
Station C	position=510	price=3.39
Station D	position=640	price=3.05

Sort these by:

```
route_position_miles
```

16. Fuel Model

Vehicle assumptions:

```
Maximum range = 500 miles
Mileage       = 10 MPG
```

Therefore:

```
Maximum fuel capacity
= 500 / 10
= 50 gallons
```

The constants should be defined centrally:

```
MAX_RANGE_MILES = 500
MILES_PER_GALLON = 10
MAX_FUEL_GALLONS = 50
```

This makes the implementation easy to modify later.

17. Fuel Optimization Algorithm

The optimization problem can be solved using a greedy strategy.

At each fuel station:

1. Determine the current amount of fuel.
2. Find the next cheaper reachable station.
3. If a cheaper station exists within 500 miles, buy only enough fuel to reach it.
4. If no cheaper reachable station exists, buy enough to maximize useful fuel, subject to the 50-gallon tank capacity.
5. Never purchase more fuel than necessary to complete the trip.

Example:

```
START
|
| 150 miles
v
Station A ($3.50)
|
| 120 miles
v
Station B ($3.10)
|
| 180 miles
v
Station C ($3.40)
|
| 100 miles
```

V
FINISH

At Station A:

Station B is cheaper.

Therefore, there is no reason to fill the entire tank at A.

Purchase enough to reach B.

At Station B:

Station C is more expensive.

Therefore, purchase enough fuel to cover as much of the upcoming journey as economically useful.

18. Greedy Algorithm — Pseudocode

```
fuel = initial_fuel
total_cost = 0

for station in candidate_stations:

    distance_from_previous = (
        station.position - previous_position
    )

    fuel -= distance_from_previous / MPG

    next_cheaper = find_next_cheaper_reachable_station(
        station
    )

    if next_cheaper:
        target_distance = (
            next_cheaper.position
            - station.position
        )
    else:
        target_distance = (
            destination_position
```

```

        - station.position
    )

    required_fuel = target_distance / MPG

    fuel_to_buy = max(
        0,
        required_fuel - fuel
    )

    fuel_to_buy = min(
        fuel_to_buy,
        MAX_FUEL_GALLONS - fuel
    )

    fuel += fuel_to_buy

    total_cost += (
        fuel_to_buy
        * station.price
    )

```

The actual implementation should additionally handle the initial station, destination, unreachable gaps, and floating-point precision.

19. Important Assumption: Initial Fuel

The assignment specifies the maximum vehicle range but does not explicitly state how much fuel the vehicle has at the beginning.

This should be explicitly documented.

The recommended implementation is to make it configurable:

```
INITIAL_FUEL_GALLONS = 50
```

This means the vehicle starts with a full tank.

The response's fuel cost then represents:

Additional fuel purchased during the journey.

Alternatively, the application can assume zero initial fuel, but this makes the start location dependent on the existence of a fuel station within the first 500 miles.

For a realistic road-trip model, starting with a full tank is the cleaner default assumption.

20. Feasibility Check

Before running the optimizer, the system should verify that the vehicle can actually complete the route.

For example:

```
START
|
| 300 miles
|
Station A
|
| 550 miles
|
Station B
```

Station B cannot be reached from Station A.

The API should not silently return an invalid route.

Instead:

```
{
  "error": {
    "code": "NO_FEASIBLE_FUEL_PLAN",
    "message": "No sequence of fuel stations can cover the route within the
vehicle's 500-mile maximum range."
  }
}
```

This is an important edge case.

21. Response Structure

A clean response could be:

```

{
  "start": {
    "address": "New York, NY",
    "latitude": 40.7128,
    "longitude": -74.0060
  },

  "finish": {
    "address": "Chicago, IL",
    "latitude": 41.8781,
    "longitude": -87.6298
  },

  "route": {
    "distance_miles": 790.4,
    "duration_minutes": 745,
    "geometry": {
      "type": "LineString",
      "coordinates": []
    }
  },

  "vehicle": {
    "max_range_miles": 500,
    "miles_per_gallon": 10
  },

  "fuel": {
    "total_gallons": 79.04,
    "total_cost": 245.72
  },

  "fuel_stops": [
    {
      "truckstop_id": 12345,
      "name": "Example Truckstop",
      "address": "123 Example Road",
      "city": "Example City",
      "state": "IL",
      "retail_price": 3.19,
      "route_position_miles": 320.4,
      "distance_from_route_miles": 1.2,
      "gallons_purchased": 20.5,
      "cost": 65.40
    }
  ]
}

```

The `geometry` can be rendered directly by a frontend using a mapping library such as Leaflet.

22. API Layer vs Business Logic

The Django view should remain thin.

Avoid putting the optimization algorithm inside `views.py`.

Recommended architecture:

```
routing/  
|  
├─ views.py  
├─ serializers.py  
├─ urls.py  
├─  
└─ services/  
    ├─ geocoding_service.py  
    ├─ routing_service.py  
    ├─ station_service.py  
    ├─ geometry_service.py  
    └─ fuel_optimizer.py
```

The view becomes approximately:

```
def post(request):  
  
    serializer = RouteRequestSerializer(  
        data=request.data  
    )  
  
    serializer.is_valid(raise_exception=True)  
  
    result = route_service.calculate_route(  
        **serializer.validated_data  
    )  
  
    return Response(result)
```

This makes the algorithm independently testable.

23. Suggested Project Structure

```
fuel-route-api/  
|  
├─ manage.py  
├─ requirements.txt  
├─ README.md  
├─ .env.example  
├─ .gitignore  
|  
├─ config/  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └── wsgi.py  
|  
├─ routing/  
│   ├── admin.py  
│   ├── apps.py  
│   ├── models.py  
│   ├── serializers.py  
│   ├── urls.py  
│   └── views.py  
|  
├─ services/  
│   ├── geocoding_service.py  
│   ├── routing_service.py  
│   ├── station_service.py  
│   ├── geometry_service.py  
│   └── fuel_optimizer.py  
|  
├─ management/  
│   └── commands/  
│       └── import_fuel_data.py  
|  
├─ tests/  
│   ├── test_api.py  
│   ├── test_optimizer.py  
│   ├── test_geometry.py  
│   └── test_station_service.py  
|  
└─ data/  
    └── fuel_prices.csv
```

24. Performance Strategy

Performance is explicitly mentioned in the assignment, so this should be part of the design.

External API calls

Target:

```
1 routing call
```

Avoid:

```
1 route call  
+  
N station routing calls
```

The fuel stations are processed locally.

Database

With fewer than 10,000 stations, a normal database query followed by local spatial processing is sufficient for this assignment.

There is no need to introduce PostGIS unless the dataset or production requirements grow significantly.

Geocoding

Fuel station coordinates are precomputed.

Only user-supplied locations need to be resolved at request time.

CSV processing

The CSV should never be parsed during an API request.

25. Caching

Caching can be added as a small optimization.

A route is determined by:

```
start coordinates
finish coordinates
```

Create a cache key such as:

```
route:<start_lat>:<start_lon>:<finish_lat>:<finish_lon>
```

Then:

```
Request
  |
  v
Cache?
/
yes    no
|      |
return ORS
      |
      v
      cache
```

This means repeated Postman requests for the same route do not repeatedly call the external routing service.

For a three-day assignment, Django's cache framework is sufficient. Redis is optional rather than necessary.

26. Testing Strategy

The most important tests should target the fuel optimizer because this is the application's core business logic.

Test 1 — Destination within range

```
Start -> Finish = 300 miles
```

The vehicle should be able to complete the trip without a fuel stop if starting with sufficient fuel.

Test 2 — Cheaper station ahead

A = \$3.50
B = \$3.00

The algorithm should avoid unnecessarily purchasing fuel at A.

Test 3 — More expensive station ahead

A = \$3.00
B = \$3.50

The optimizer should use the cheaper fuel at A where feasible.

Test 4 — Exactly 500 miles

A station exactly 500 miles away should be considered reachable.

Test 5 — More than 500 miles

A station 501 miles away should not be considered reachable from an empty/appropriate fuel state.

Test 6 — Station outside route corridor

A station 20 miles from the route should not be considered when the corridor is 5 miles.

Test 7 — No feasible fuel plan

The API should return an appropriate error.

Test 8 — Duplicate stations

The import process should not create duplicate records for the same OPIS Truckstop ID.

27. Error Handling

External API failures should not result in raw exceptions being returned to the client.

Handle:

Invalid location
Location outside USA

```
No route
Routing API timeout
Routing API rate limit
Invalid fuel data
No nearby stations
No feasible fuel plan
```

Example:

```
{
  "error": {
    "code": "ROUTING_SERVICE_UNAVAILABLE",
    "message": "Unable to calculate the route at this time."
  }
}
```

The external API key should be stored in an environment variable:

```
ORS_API_KEY=...
```

and never committed to Git.

28. Security

The API should:

- Keep external API keys in environment variables.
- Validate request payloads.
- Set reasonable request timeouts for external services.
- Avoid exposing internal exceptions.
- Avoid committing the real `.env` file.
- Include `.env.example` in the repository.

Example:

```
.env
.env.example
```

```
.env.example:
```

```
ORS_API_KEY=
```

29. Why Not PostGIS?

PostGIS would be an excellent production solution for large geospatial datasets.

However, for this assignment:

```
Fuel stations < 10,000
```

so the complexity is unnecessary.

A simpler approach is:

```
Database
  ↓
load station coordinates
  ↓
Shapely geometry calculations
```

This is easier to implement, easier to test, and easier to explain in the five-minute Loom.

If the system later grows to hundreds of thousands or millions of stations, PostGIS becomes much more attractive.

30. Why Not Call a Map API for Every Station?

Because the routing API is needed only to determine the primary journey.

Once the route geometry is available:

```
Route geometry
  +
Station coordinates
  ↓
local geometry calculations
```

This eliminates thousands of external calls.

This is probably the most important architectural decision in the assignment.

31. Complexity

Suppose:

```
N = number of fuel stations
M = number of route coordinates
```

A straightforward implementation can process fewer than 10,000 stations locally.

The main goal is to avoid:

```
N × external API calls
```

Instead:

```
1 × external API call
+
local processing
```

For the assignment's dataset size, this is an appropriate trade-off.

If profiling shows geometry processing to be expensive, we can optimize the route representation or use spatial indexing, but this should be driven by measurements rather than premature complexity.

32. Important Definition of "Optimal"

The assignment says that optimal "mostly means cost effective based on fuel prices."

Therefore, the implementation should explicitly define optimality as:

Minimize the total cost of fuel purchased while satisfying the vehicle's maximum range constraint and selecting fuel stations within the permitted route corridor.

The algorithm should not attempt to optimize:

- Driving time
- Traffic

- Distance to station
- Number of stops

unless these are introduced as additional constraints.

This keeps the solution deterministic and aligned with the assignment.

33. Optional Future Improvement: Detour Cost

A more advanced version could account for the actual cost of leaving the route.

For example:

```
Station A
1 mile detour
$3.50

Station B
8 mile detour
$3.20
```

The cheapest fuel is not necessarily the cheapest overall option after considering the additional driving.

However, implementing this correctly would require additional routing calls or a more sophisticated routing model.

Therefore, it should **not be part of the initial implementation**.

The initial version should use:

```
fuel price
+
route corridor
```

as the primary criteria.

34. Optional Map Frontend

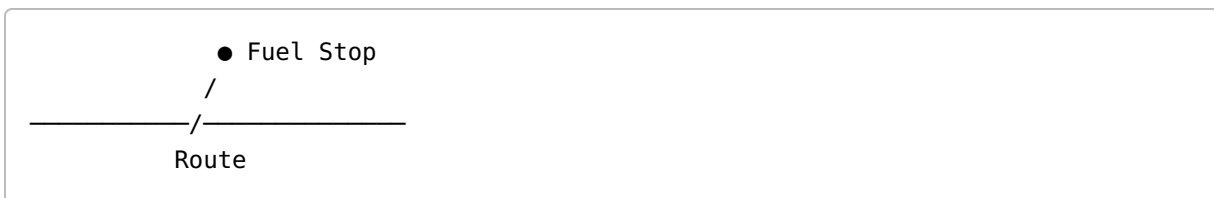
The assignment asks for the API to return a map/route.

The backend should return GeoJSON rather than generating an image.

For example:

```
{
  "type": "LineString",
  "coordinates": [
    [-74.0060, 40.7128],
    [-73.90, 40.80],
    ...
  ]
}
```

A frontend can then display:



A simple Leaflet frontend can consume the GeoJSON and add markers for recommended fuel stops.

For the assignment, however, the API + Postman demonstration is sufficient unless they specifically require a frontend.

35. Recommended Development Sequence

The project should be implemented in this order.

Step 1

Create Django project.

Step 2

Create `FuelStation` model.

Step 3

Create database migration.

Step 4

Implement CSV import command.

Step 5

Resolve and store fuel-station coordinates.

Step 6

Implement location resolution.

Step 7

Implement OpenRouteService client.

Step 8

Return route geometry and distance.

Step 9

Implement route/station spatial matching.

Step 10

Calculate each station's position along the route.

Step 11

Implement fuel optimization.

Step 12

Combine everything into the API.

Step 13

Add validation and error handling.

Step 14

Add caching.

Step 15

Write tests.

Step 16

Prepare Postman collection.

Step 17

Record Loom.

36. Suggested Implementation Timeline

Because the assignment gives three days, the work can be divided as follows.

Day 1 — Core implementation

```
Django setup
Database model
CSV import
Geocoding
OpenRouteService integration
Basic API
```

Goal:

```
POST request
  ↓
Route
  ↓
JSON response
```

Day 2 — Optimization

Implement:

```
Station filtering
Route projection
Route positions
Fuel optimizer
Total cost
Edge cases
```

Then write unit tests.

Goal:

```
POST request
  ↓
Route
  ↓
Candidate stations
  ↓
Optimal fuel stops
  ↓
Total cost
```

Day 3 — Quality

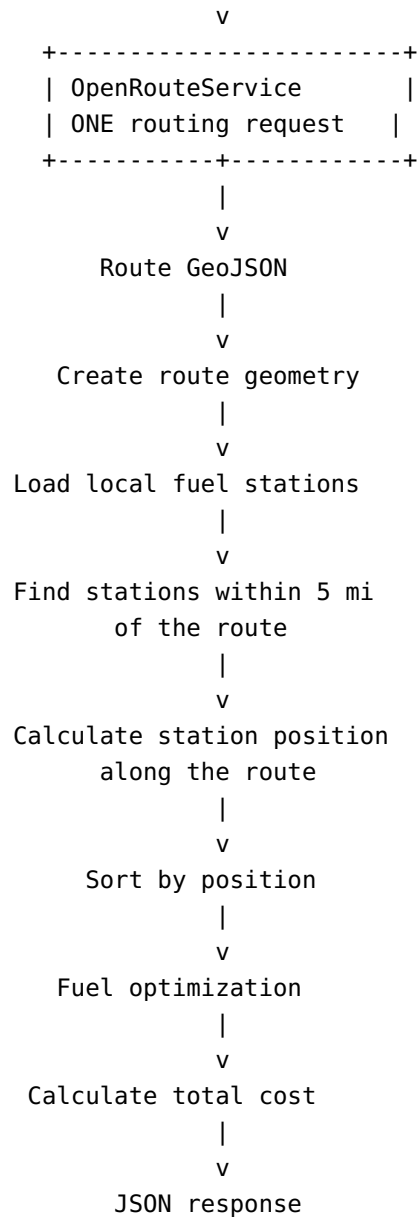
Focus on:

```
Performance
Caching
Error handling
README
Postman collection
Code cleanup
Deployment
Loom recording
```

37. Final API Flow

The complete request should look like this:

```
      POST
    /api/v1/route/
      |
      v
  Validate request
      |
      v
  Resolve locations
      |
```



38. Final Recommended Stack

Language
Python

Framework
Django 6.1.1
Django REST Framework

Database
SQLite initially
PostgreSQL optional

Routing
OpenRouteService

Geometry
Shapely
PyProj

HTTP
httpx

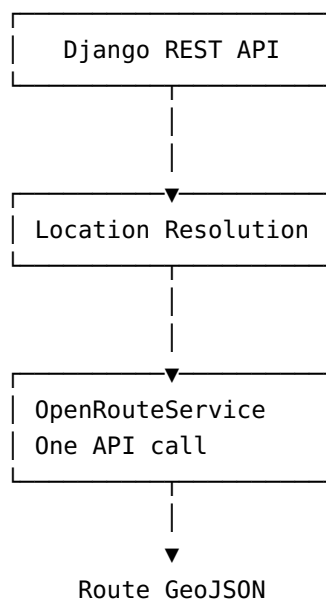
Testing
Django Test Framework / pytest

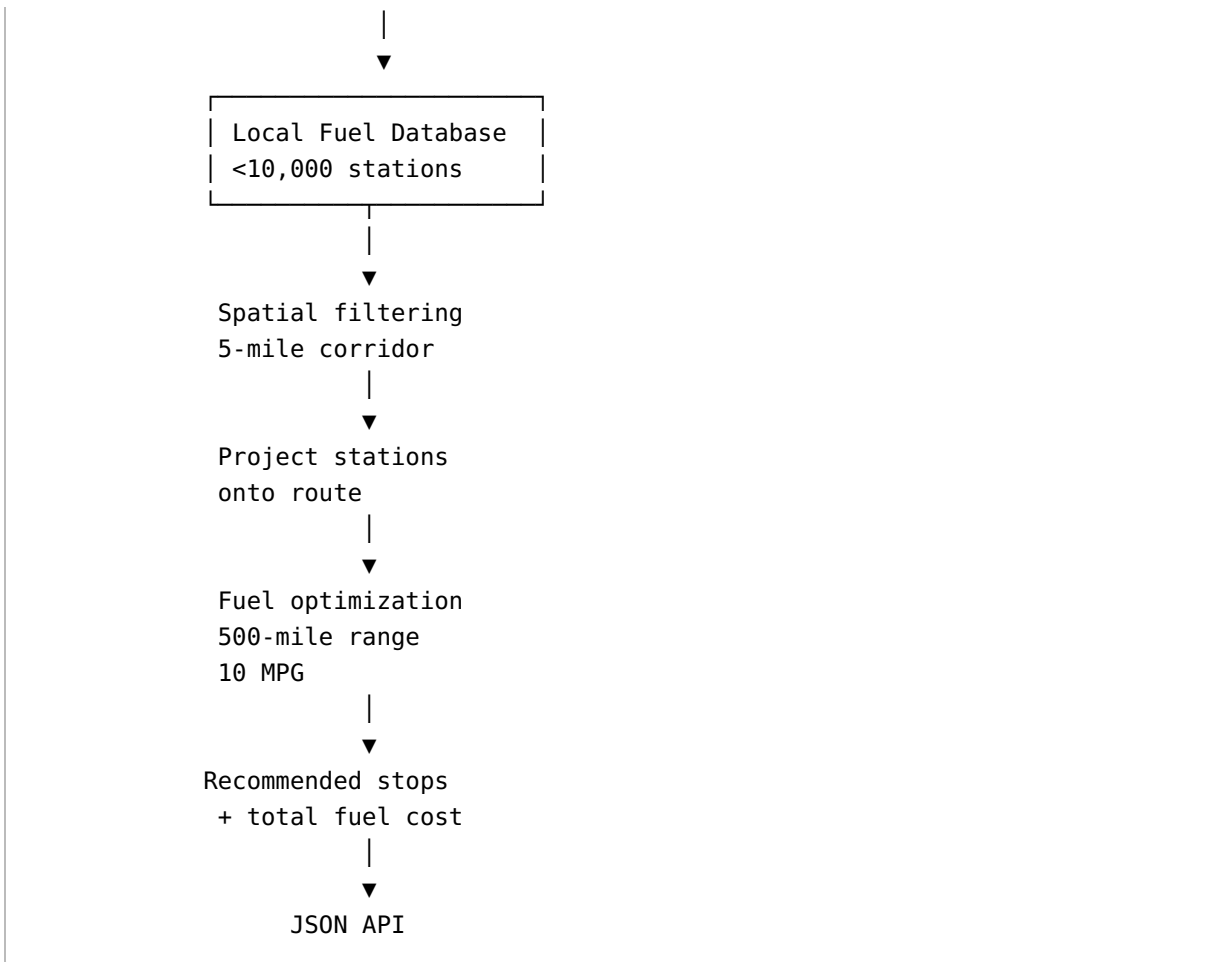
Caching
Django cache framework

API Documentation
README + Postman collection

39. Final Architecture Decision

The most appropriate implementation for this assignment is therefore:





This approach satisfies the assignment while keeping the implementation **simple enough to complete in three days, fast enough for the expected dataset, easy to test, and easy to explain during the Loom.**

The most important engineering decisions are:

1. **Preprocess the fuel CSV once.**
2. **Store station coordinates locally.**
3. **Call the routing API only once per uncached request.**
4. **Perform station/route matching locally.**
5. **Convert the geographic problem into a 1-dimensional fuel optimization problem.**
6. **Use a greedy fuel-purchasing algorithm under the 500-mile range constraint.**
7. **Return GeoJSON so the route can be rendered directly on a map.**
8. **Keep the Django view thin and put the business logic in services.**
9. **Make assumptions such as initial fuel and route-corridor width explicit.**
10. **Test the optimizer independently from Django and external APIs.**